Step 1 - Read the JSON file using the spark dataframe reader API

```
dbutils.widgets.text("p_data_source", "")
v_data_source = dbutils.widgets.get("p_data_source")
```

```
dbutils.widgets.text("p_file_date", "2021-03-28")
v_file_date = dbutils.widgets.get("p_file_date")
```

```
%run "../includes/configuration"
```

```
%run "../includes/common_functions"
```

```
from pyspark.sql.types import StructType, StructField, IntegerType, StringType, FloatType
```

```
results_schema = StructType(fields=[StructField("resultId", IntegerType(), False),
                                    StructField("raceId", IntegerType(), True),
                                    StructField("driverId", IntegerType(), True),
                                    StructField("constructorId", IntegerType(), True),
                                    StructField("number", IntegerType(), True),
                                    StructField("grid", IntegerType(), True),
                                    StructField("position", IntegerType(), True),
                                    StructField("positionText", StringType(), True),
                                    StructField("positionOrder", IntegerType(), True),
                                    StructField("points", FloatType(), True),
                                    StructField("laps", IntegerType(), True),
                                    StructField("time", StringType(), True),
                                    StructField("milliseconds", IntegerType(), True),
                                    StructField("fastestLap", IntegerType(), True),
                                    StructField("rank", IntegerType(), True),
                                    StructField("fastestLapTime", StringType(), True),
                                    StructField("fastestLapSpeed", FloatType(), True),
                                    StructField("statusId", StringType(), True)])
```

```
results_df = spark.read \
.schema(results_schema) \
.json(f"{raw_folder_path}/{v_file_date}/results.json")
```

```
display(results_df)
```

Table



	resultId	raceId	driverId	constructorId	number	grid	position	positionTe
1	1	18	1	1	22	1	1	1
2	2	18	2	2	3	5	2	2
3	3	18	3	3	7	7	3	3
4	4	18	4	4	5	11	4	4
5	5	18	5	1	23	3	5	5
6	6	18	6	3	8	13	6	6
7	7	18	7	5	14	17	7	7
8	8	18	8	6	1	15	8	8
9	9	18	9	2	4	2	null	R
10	10	18	10	7	12	18	null	R
11	11	18	11	8	18	19	null	R
12	12	18	12	4	6	20	null	R
13	13	18	13	6	2	4	null	R
14	14	18	14	9	9	8	null	R

10,000+ rows | Truncated data due to row limit

Step 2 - Rename columns and add new columns

```
from pyspark.sql.functions import current_timestamp, lit
```

```
results_with_columns_df = results_df.withColumnRenamed("resultId", "result_id") \
                                    .withColumnRenamed("raceId", "race_id") \
                                    .withColumnRenamed("driverId", "driver_id") \
                                    .withColumnRenamed("constructorId", "constructor_id") \
                                    .withColumnRenamed("positionText", "position_text") \
                                    .withColumnRenamed("positionOrder", "position_order") \
                                    .withColumnRenamed("fastestLap", "fastest_lap") \
                                    .withColumnRenamed("fastestLapTime", "fastest_lap_time") \
                                    .withColumnRenamed("fastestLapSpeed", "fastest_lap_speed") \
                                    .withColumn("data_source", lit(v_data_source)) \
                                    .withColumn("file_date", lit(v_file_date))
```

```
results_with_ingestion_date_df = add_ingestion_date(results_with_columns_df)
```

```
display(results_with_columns_df)
```

Table

	result_id	race_id	driver_id	constructor_id	number	grid	position	positio
1	1	18	1	1	22	1	1	1
2	2	18	2	2	3	5	2	2
3	3	18	3	3	7	7	3	3
4	4	18	4	4	5	11	4	4
5	5	18	5	1	23	3	5	5
6	6	18	6	3	8	13	6	6
7	7	18	7	5	14	17	7	7
8	8	18	8	6	1	15	8	8
9	9	18	9	2	4	2	null	R

10	10	18	10	7	12	18	null	R
11	11	18	11	8	18	19	null	R
12	12	18	12	4	6	20	null	R
13	13	18	13	6	2	4	null	R
14	14	18	14	9	9	8	null	R

10,000+ rows | Truncated data due to row limit

Step 3 - Drop the unwanted column

```
from pyspark.sql.functions import col
```

```
results_final_df = results_with_columns_df.drop(col("statusId"))
```

```
display(results_final_df)
```

Table

	result_id	race_id	driver_id	constructor_id	number	grid	position	positio
1	1	18	1	1	22	1	1	1
2	2	18	2	2	3	5	2	2
3	3	18	3	3	7	7	3	3
4	4	18	4	4	5	11	4	4
5	5	18	5	1	23	3	5	5
6	6	18	6	3	8	13	6	6
7	7	18	7	5	14	17	7	7
8	8	18	8	6	1	15	8	8
9	9	18	9	2	4	2	null	R
10	10	18	10	7	12	18	null	R
11	11	18	11	8	18	19	null	R
12	12	18	12	4	6	20	null	R
13	13	18	13	6	2	4	null	R
14	14	18	14	9	9	8	null	R

10,000+ rows | Truncated data due to row limit

Step 4 - Write to output to processed container in parquet format

Method 1

```
# for race_id_list in results_final_df.select("race_id").distinct().collect():
#    if (spark._jsparkSession.catalog().tableExists("f1_processed.results")):
#      spark.sql(f"ALTER TABLE f1_processed.results DROP IF EXISTS PARTITION (race_id = {race_id_list.race_id})")
```

```
#results_final_df.write.mode("overwrite").partitionBy('race_id').format("parquet").saveAsTable("f1_processed.results")
```

Method 2

```
# overwrite_partition(results_final_df, 'f1_processed', 'results', 'race_id')
```

```
results_deduped_df = results_final_df.dropDuplicates(['race_id', 'driver_id'])
```

```
merge_condition = "tgt.result_id = src.result_id AND tgt.race_id = src.race_id"
merge_delta_data(results_deduped_df, 'f1_processed', 'results', processed_folder_path, merge_condition, 'race_id')
```

```
%sql
SELECT race_id, COUNT(1)
FROM f1_processed.results
GROUP BY race_id
ORDER BY race_id DESC;
```

Table

	race_id	count(1)
1	1053	20
2	1052	20
3	1047	20
4	1046	20
5	1045	20
6	1044	20
7	1043	20
8	1042	20
9	1041	20
10	1040	20
11	1039	20
12	1038	20
13	1037	20
14	1036	20
15	1035	20

1,037 rows

```
%sql
select race_id, driver_id, count(1)
from f1_processed.results
group by race_id, driver_id
having count(1) > 1
order by race_id, driver_id desc;
```

Table

	race_id	driver_id	count(1)

1	540	229	2
2	717	373	2
3	733	465	2
4	742	475	2
5	745	418	2
6	746	475	2
7	766	541	2
8	770	479	2
9	774	566	2
10	776	581	2
11	777	578	2
12	777	501	2
13	779	356	2
14	780	581	2
15	780	479	2

85 rows

```
# display(spark.read.parquet(f"{processed_folder_path}/results"))
```

```
%sql
select * from f1_processed.results;
```

Table

	result_id	race_id	driver_id	constructor_id	number	grid	position	positio
1	20327	337	3	131	4	5	5	5
2	20337	337	5	205	19	21	15	15
3	20333	337	9	4	11	9	11	11
4	20334	337	16	10	14	10	12	12
5	20329	337	18	1	1	8	7	7
6	20326	337	20	9	5	1	4	4
7	20328	337	30	131	3	7	6	6
8	20336	337	807	3	10	13	14	14
9	20345	337	810	166	25	22	null	R
10	20341	337	811	164	21	23	null	R
11	20325	337	1	1	2	4	3	3
12	20323	337	4	6	8	3	1	1
13	20342	337	10	166	24	19	null	R
14	20324	337	13	6	7	2	2	2

10,000+ rows | Truncated data due to row limit

Success